

```

1 # 한글 폰트 문제 해결을 위한 설정 코드
2 import matplotlib.font_manager as fm
3 import platform
4 import os
5
6 # 광고 클릭 예측 데이터 분석 - Part 1
7 # 타겟 분석 + 시간 패턴 + 사용자 세그먼트 분석
8
9 import pandas as pd
10 import numpy as np
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13 from datetime import datetime
14 import warnings
15 warnings.filterwarnings('ignore')
16
17 # # 한글 폰트 설정 (선택사항)
18 # plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial Unicode MS', 'Malgun Gothic']
19 # plt.rcParams['axes.unicode_minus'] = False
20
21 # 기본 스타일 설정
22 plt.style.use('default')
23 sns.set_palette('husl')
24
25 def setup_korean_font():
26     """운영체제별 한글 폰트 설정"""
27
28     print("🔍 현재 시스템:", platform.system())
29     print("🔍 사용 가능한 폰트 확인 중...")
30
31     # 운영체제별 한글 폰트 리스트
32     korean_fonts = {
33         'Windows': [
34             'Malgun Gothic',
35             'Microsoft Yohhei',
36             'SimHei',
37             'NanumGothic',
38             'NanumBarunGothic'
39         ],
40         'Darwin': [ # macOS
41             'AppleGothic',
42             'Apple SD Gothic Neo',
43             'NanumGothic',
44             'NanumBarunGothic'
45         ],
46         'Linux': [
47             'DejaVu Sans',
48             'NanumGothic',
49             'NanumBarunGothic',
50             'UnDotum'
51         ]
52     }
53
54     # 시스템에 설치된 폰트 목록
55     available_fonts = [f.name for f in fm.fontManager.ttflist]
56
57     # 운영체제에 맞는 한글 폰트 찾기
58     system = platform.system()
59     fonts_to_try = korean_fonts.get(system, korean_fonts['Linux'])
60
61     found_font = None
62     for font in fonts_to_try:
63         if font in available_fonts:
64             found_font = font
65             break
66
67     if found_font:
68         plt.rcParams['font.family'] = found_font
69         plt.rcParams['axes.unicode_minus'] = False
70         print(f"✅ 한글 폰트 설정 완료: {found_font}")
71         return True
72     else:
73         print("❌ 적절한 한글 폰트를 찾을 수 없습니다.")
74         print("📦 설치된 폰트 중 일부:")
75         print([f for f in available_fonts if any(keyword in f.lower() for keyword in ['gothic', 'nanum', 'malgun'])][:10])
76         return False
77
78 def test_korean_display():
79     """한글 출력 테스트"""
80     import matplotlib.pyplot as plt
81     import numpy as np
82
83     fig, ax = plt.subplots(figsize=(8, 6))
84
85     # 테스트 데이터
86     x = ['일', '화', '수', '목', '금', '토', '일']
87     y = np.random.rand(7)
88
89     ax.bar(x, y)
90     ax.set_title('한글 폰트 테스트')
91     ax.set_xlabel('요일')
92     ax.set_ylabel('값')
93
94     plt.tight_layout()
95     plt.show()
96
97     print("위 차트에서 한글이 정상적으로 보이나요?")
98
99 setup_korean_font()
100 # # 폰트 설정 실행
101 # if setup_korean_font():
102 #     test_korean_display()
103 # else:
104 #     print("\n💡 해결 방법:")
105 #     print("1. 나눔고딕 폰트 설치: https://hangeul.naver.com/2017/nanum")
106 #     print("2. 영어 버전 코드 사용")
107 #     print("3. 폰트 캐시 초기화: matplotlib 폰트 캐시 삭제")
108
109 # # 폰트 캐시 초기화 함수 (필요시 사용)
110 def clear_matplotlib_cache():
111     """matplotlib 폰트 캐시 초기화"""
112     try:
113         import matplotlib
114         cache_dir = matplotlib.get_cachedir()
115         print(f"캐시 디렉토리: {cache_dir}")
116
117         # 캐시 파일들 삭제
118         for file in os.listdir(cache_dir):
119             if file.endswith('.cache'):
120                 os.remove(os.path.join(cache_dir, file))
121                 print(f"삭제됨: {file}")
122
123         print("✅ 폰트 캐시 초기화 완료")
124         print("🔄 Python 재시작 후 다시 시도해주세요")
125
126     except Exception as e:
127         print(f"❌ 캐시 초기화 실패: {e}")
128
129 # # 필요시 캐시 초기화 실행
130 # clear_matplotlib_cache()

```

 현재 시스템: Windows
 사용 가능한 폰트 확인 중...
 한글 폰트 설정 완료: Malgun Gothic
 True

1 코딩을 시작하거나 AI로 코드를 생성하세요.

```
1 print("🔴 광고 클릭 예측 데이터 분석 시작!")
2 print("w" * 60)
3
4 # =====
5 # 📄 1. 데이터 로드 및 기본 정보
6 # =====
7
8 # 데이터 로드
9 df = pd.read_parquet('./train.parquet')
10
11 print("📄 데이터 기본 정보:")
12 print(f"총 샘플 수: {len(df):,}")
13 print(f"총 컬럼 수: {len(df.columns):,}")
14 print(f"메모리 사용량: {df.memory_usage(deep=True).sum() / 1024*2:.2f} MB")
15
16 # 결측값 확인
17 missing_info = df.isnull().sum()
18 missing_info = missing_info[missing_info > 0].sort_values(ascending=False)
19 if len(missing_info) > 0:
20     print(f"🚩 결측값이 있는 컬럼: {len(missing_info)}개")
21     print(missing_info.head(10))
22 else:
23     print(f"🟢 결측값 없음")
24
25 print(f"📅 데이터 타입 분포:")
26 print(df.dtypes.value_counts())
27 print(df.info())
28 display(df.describe())
```

🔴 광고 클릭 예측 데이터 분석 시작!

=====

📄 데이터 기본 정보:
총 샘플 수: 10,784,179
총 컬럼 수: 119
메모리 사용량: 27897.43 MB

🚩 결측값이 있는 컬럼: 96개

```
feat_e_3    1085557
feat_a_8      18598
feat_a_15     18598
feat_a_17     18598
feat_a_1      18598
feat_a_2      18598
feat_a_3      18598
feat_a_4      18598
feat_a_5      18598
feat_a_6      18598
dtype: int64
```

📅 데이터 타입 분포:

```
float32    112
object       6
int32        1
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10784179 entries, 0 to 10784178
Columns: 119 entries, gender to clicked
dtypes: float32(112), int32(1), object(6)
memory usage: 5.0+ GB
None
```

	l_feat_1	l_feat_2	l_feat_3	l_feat_4	l_feat_5	l_feat_6	l_feat_7	l_feat_8	l_feat_9	l_feat_10	...	history_b_22	history_b_23	history_b_24	history_b_25	history_b_26	history_b_27	h
count	1.070418e+07	1.068697e+07	1.070418e+07	1.070418e+07	1.070418e+07	1.070418e+07	1.070418e+07	1.068697e+07	1.070418e+07	1.070418e+07	...	1.068697e+07	1.068697e+07	1.068697e+07	1.068697e+07	1.068697e+07	1.068697e+07	1
mean	1.858145e+00	1.831941e+00	2.357217e+00	9.543344e+00	3.938107e+02	3.147881e+02	1.479098e+02	1.994026e+00	2.128027e+02	1.239446e+02	...	4.416594e-01	4.396587e-01	7.326673e-02	2.928778e-02	2.777950e-01	3.167009e-01	9
std	3.368007e-01	3.562318e-01	7.160997e-01	5.882460e+00	2.896129e+02	2.312981e+02	7.103545e+01	7.684457e-02	1.244261e+02	6.197770e+01	...	1.964987e+00	1.964032e+00	3.271963e-01	1.309372e-01	1.222967e+00	1.397911e+00	4
min	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	...	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0
25%	2.000000e+00	2.000000e+00	2.000000e+00	7.000000e+00	1.050000e+02	1.090000e+02	9.400000e+01	2.000000e+00	1.010000e+02	8.300000e+01	...	5.310000e-02	5.250600e-02	8.751000e-03	3.484200e-03	3.454640e-02	3.886000e-02	1
50%	2.000000e+00	2.000000e+00	2.000000e+00	7.000000e+00	3.760000e+02	2.780000e+02	1.460000e+02	2.000000e+00	2.240000e+02	1.250000e+02	...	9.345600e-02	9.240300e-02	1.540050e-02	6.135000e-03	6.100000e-02	6.874240e-02	2
75%	2.000000e+00	2.000000e+00	3.000000e+00	1.200000e+01	6.520000e+02	4.710000e+02	2.120000e+02	2.000000e+00	3.090000e+02	1.790000e+02	...	2.088180e-01	2.064240e-01	3.440400e-02	1.372980e-02	1.342936e-01	1.527424e-01	4
max	2.000000e+00	2.000000e+00	3.000000e+00	2.600000e+01	1.070000e+03	9.030000e+02	3.130000e+02	2.000000e+00	4.760000e+02	2.620000e+02	...	4.500000e+01	4.500000e+01	7.500000e+00	3.000000e+00	2.800000e+01	3.200000e+01	1
8 rows x 113 columns																		

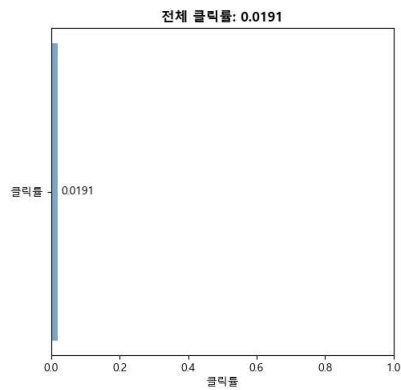
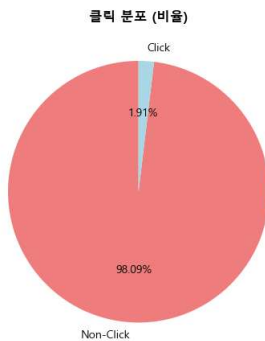
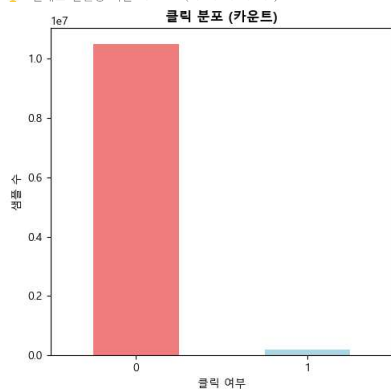
```
1
2 # =====
3 # 🖱️ 2. 타겟 변수 (clicked) 분석
4 # =====
5
6 print("\n" + "="*60)
7 print("🖱️ 타겟 변수 (clicked) 분석")
8 print("="*60)
9
10 # 클릭률 계산
11 click_rate = df['clicked'].mean()
12 total_clicks = df['clicked'].sum()
13 total_samples = len(df)
14
15 print(f"📊 전체 클릭률: {click_rate:.4f} ({click_rate*100:.2f}%)")
16 print(f"📊 총 클릭 수: {total_clicks:,} / {total_samples:,}")
17 print(f"📊 클래스 불균형 비율: {(1-click_rate)/click_rate:.1f}:1 (Non-click:Click)")
18
19 # 타겟 분포 시각화
20 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
21
22 # 클릭 분포 (카운트)
23 df['clicked'].value_counts().plot(kind='bar', ax=axes[0], color=['lightcoral', 'lightblue'])
24 axes[0].set_title('클릭 분포 (카운트)', fontsize=12, fontweight='bold')
25 axes[0].set_xlabel('클릭 여부')
26 axes[0].set_ylabel('샘플 수')
27 axes[0].tick_params(axis='x', rotation=0)
28
29 # 클릭 분포 (비율) - 파이차트
30 sizes = df['clicked'].value_counts()
31 labels = ['Non-Click', 'Click']
32 colors = ['lightcoral', 'lightblue']
33 axes[1].pie(sizes, labels=labels, colors=colors, autopct='%1.2f%%', startangle=90)
34 axes[1].set_title('클릭 분포 (비율)', fontsize=12, fontweight='bold')
35
36 # 클릭률 시각화 (게이지 형태)
37 axes[2].barh(['클릭률'], [click_rate], color='steelblue', alpha=0.7)
38 axes[2].set_xlim(0, 1)
39 axes[2].set_title(f'전체 클릭률: {click_rate:.4f}', fontsize=12, fontweight='bold')
40 axes[2].set_xlabel('클릭률')
41 for i, v in enumerate([click_rate]):
42     axes[2].text(v + 0.01, i, f'{v:.4f}', va='center')
43
44 plt.tight_layout()
45 plt.show()
```

타겟 변수 (clicked) 분석

전체 클릭률: 0.0191 (1.91%)

총 클릭 수: 204,179 / 10,704,179

클래스 불균형 비율: 51:1 (Non-Click:Click)



```
1 # 타임 정리: 문자열 -> 숫자(가능한 경우), 클릭도 숫자 보강
2 for col in ['hour', 'day_of_week', 'clicked']:
3     if col in df.columns:
4         df[col] = pd.to_numeric(df[col], errors='coerce')
5
6 # 요일이 1~7로 들어왔다면 0~6로 보정
7 if 'day_of_week' in df.columns:
8     s = df['day_of_week'].dropna()
9     if len(s) and s.between(1, 7).all():
10         df['day_of_week'] = df['day_of_week'] - 1
11 # 정수(결측 허용)로 캐스팅
12 df['day_of_week'] = df['day_of_week'].astype('Int64')
13
14 # 필요 컬럼의 NaN 제거(선택)
15 need_cols = [c for c in ['hour', 'day_of_week', 'clicked'] if c in df.columns]
16 if need_cols:
17     df = df.dropna(subset=need_cols)
18
19
20 dow_map = {
21     'Mon':0, 'Tue':1, 'Wed':2, 'Thu':3, 'Fri':4, 'Sat':5, 'Sun':6,
22     '일':0, '화':1, '수':2, '목':3, '금':4, '토':5, '일':6
23 }
24 if 'day_of_week' in df.columns and df['day_of_week'].dtype == 'object':
25     df['day_of_week'] = df['day_of_week'].map(dow_map)
```

```
1
2 # =====
3 # 📊 3. 시간 패턴 분석
4 # =====
5
6 print("\n" + "="*60)
7 print("📊 시간 패턴 분석")
8 print("="*60)
9
10 # 시간대별 활동 패턴
11 hourly_stats = df.groupby('hour').agg({
12     'clicked': ['count', 'sum', 'mean']
13 }).round(4)
14 hourly_stats.columns = ['총_샘플수', '클릭수', '클릭률']
15
16 print("📊 시간대별 통계 (상위 5개):")
17 print(hourly_stats.sort_values('클릭률', ascending=False).head())
18
19 # 요일별 활동 패턴
20 df['day_of_week'] -= 1
21 daily_stats = df.groupby('day_of_week').agg({
22     'clicked': ['count', 'sum', 'mean']
23 }).round(4)
24 daily_stats.columns = ['총_샘플수', '클릭수', '클릭률']
25
26 print(f"\n📊 요일별 통계:")
27 day_names = ['일', '화', '수', '목', '금', '토', '일']
28 daily_stats.index = [day_names[i] if i < len(day_names) else f'Day_{i}'] for i in daily_stats.index]
29 print(daily_stats)
30
31 # 시간 패턴 시각화
32 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
33
34 # 1. 시간대별 샘플 수
35 hourly_stats['총_샘플수'].plot(kind='bar', ax=axes[0,0], color='skyblue', alpha=0.8)
36 axes[0,0].set_title('시간대별 총 샘플 수', fontsize=12, fontweight='bold')
37 axes[0,0].set_xlabel('시간')
38 axes[0,0].set_ylabel('샘플 수')
39 axes[0,0].tick_params(axis='x', rotation=45)
40
41 # 2. 시간대별 클릭률
42 hourly_stats['클릭률'].plot(kind='line', ax=axes[0,1], color='red', marker='o', linewidth=2)
43 axes[0,1].set_title('시간대별 클릭률 추이', fontsize=12, fontweight='bold')
44 axes[0,1].set_xlabel('시간')
45 axes[0,1].set_ylabel('클릭률')
46 axes[0,1].grid(True, alpha=0.3)
47
48 # 3. 요일별 샘플 수
49 daily_stats['총_샘플수'].plot(kind='bar', ax=axes[0,2], color='lightgreen', alpha=0.8)
50 axes[0,2].set_title('요일별 총 샘플 수', fontsize=12, fontweight='bold')
51 axes[0,2].set_xlabel('요일')
52 axes[0,2].set_ylabel('샘플 수')
53 axes[0,2].tick_params(axis='x', rotation=45)
54
55 # 4. 요일별 클릭률
56 daily_stats['클릭률'].plot(kind='line', ax=axes[1,0], color='purple', marker='s', linewidth=2)
57 axes[1,0].set_title('요일별 클릭률 추이', fontsize=12, fontweight='bold')
58 axes[1,0].set_xlabel('요일')
59 axes[1,0].set_ylabel('클릭률')
60 axes[1,0].grid(True, alpha=0.3)
61
62 # 5. 시간 x 요일 히트맵 (클릭률)
63 pivot_table = df.groupby(['day_of_week', 'hour'])['clicked'].mean().unstack()
64 im = axes[1,1].imshow(pivot_table.values, cmap='YlOrRd', aspect='auto')
65 axes[1,1].set_title('시간 x 요일 클릭률 히트맵', fontsize=12, fontweight='bold')
66 axes[1,1].set_xlabel('시간')
67 axes[1,1].set_ylabel('요일')
68 axes[1,1].set_yticks(range(len(pivot_table.index)))
69 axes[1,1].set_yticklabels([day_names[i] if i < len(day_names) else f'Day_{i}'] for i in pivot_table.index])
70 plt.colorbar(im, ax=axes[1,1], label='클릭률')
71
72 # 6. 피크 시간대 분석
73 peak_hours = hourly_stats.nlargest(5, '클릭률')
74 peak_hours.index.name = '시간'
75 peak_hours['클릭률'].plot(kind='bar', ax=axes[1,2], color='orange', alpha=0.8)
76 axes[1,2].set_title('클릭률 TOP 5 시간대', fontsize=12, fontweight='bold')
77 axes[1,2].set_xlabel('시간')
```

```
78 axes[1,2].set_ylabel('클릭률')
79 axes[1,2].tick_params(axis='x', rotation=45)
80
81 plt.tight_layout()
82 plt.show()
83
84 # 시간 인사이트 출력
85 print(f"\n🕒 시간 패턴 인사이트:")
86 peak_hour = hourly_stats['클릭률'].idxmax()
87 low_hour = hourly_stats['클릭률'].idxmin()
88 print(f" 최고 클릭률 시간: {peak_hour}시 ({hourly_stats.loc[peak_hour, '클릭률']:.4f})")
89 print(f" 최저 클릭률 시간: {low_hour}시 ({hourly_stats.loc[low_hour, '클릭률']:.4f})")
90
91 peak_day = daily_stats['클릭률'].idxmax()
92 low_day = daily_stats['클릭률'].idxmin()
93 print(f" 최고 클릭률 요일: {peak_day} ({daily_stats.loc[peak_day, '클릭률']:.4f})")
94 print(f" 최저 클릭률 요일: {low_day} ({daily_stats.loc[low_day, '클릭률']:.4f})")
95
```

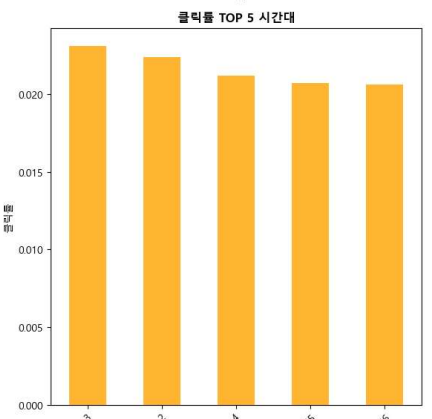
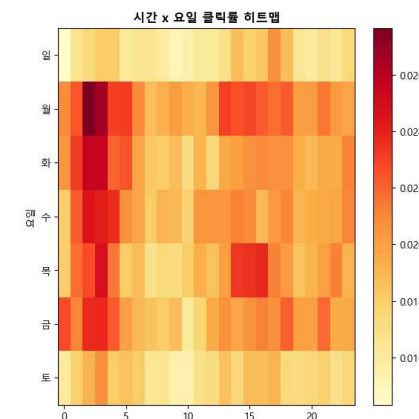
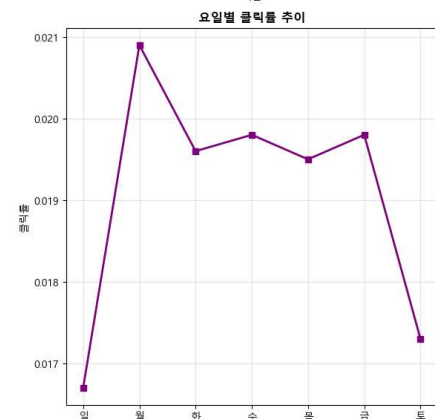
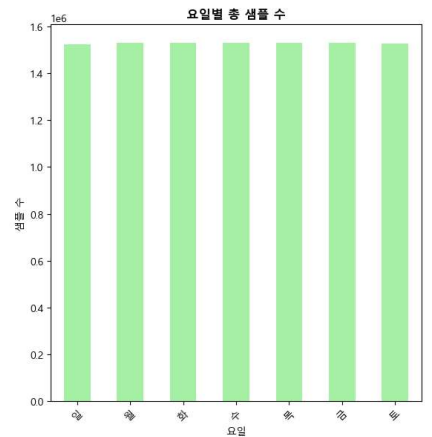
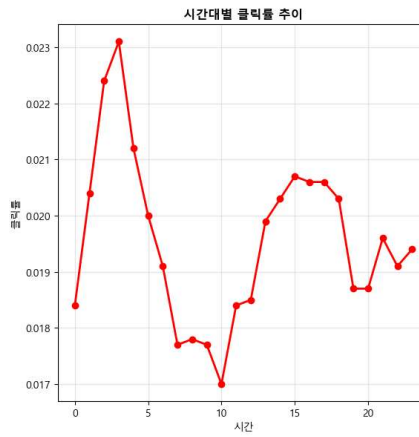
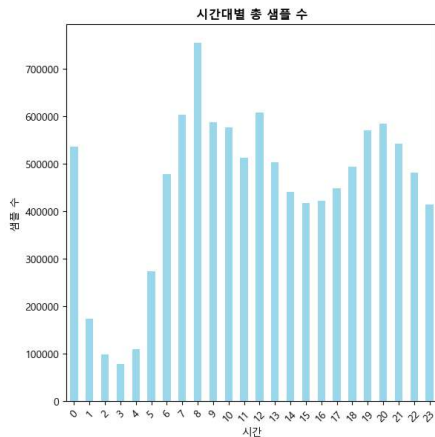
🕒 시간 패턴 분석

📊 시간대별 통계 (상위 5개):

	총_샘플수	클릭수	클릭률
hour			
3	78649	1820	0.0231
2	98671	2214	0.0224
4	118006	2337	0.0212
15	417579	8645	0.0207
16	421292	8662	0.0206

📊 요일별 통계:

	총_샘플수	클릭수	클릭률
일	1525483	25483	0.0167
월	1532022	32022	0.0209
화	1529921	29921	0.0196
수	1530297	30297	0.0198
목	1529839	29839	0.0195
금	1530282	30282	0.0198
토	1526335	26335	0.0173



```
1
2 #
3 # 📊 4. 사용자 세그먼트 분석
4 #
5
6 print("\n" + "="*60)
7 print("👤 사용자 세그먼트 분석")
8 print("="*60)
9
10 # 성별 분석
11 gender_stats = df.groupby('gender').agg({
12     'clicked': ['count', 'sum', 'mean']
13 }).round(4)
14 gender_stats.columns = ['총_샘플수', '클릭수', '클릭률']
15
16 print("📊 성별 통계:")
17 print(gender_stats)
18
19 # 연령그룹 분석
20 age_stats = df.groupby('age_group').agg({
21     'clicked': ['count', 'sum', 'mean']
22 }).round(4)
23 age_stats.columns = ['총_샘플수', '클릭수', '클릭률']
24
25 print(f"\n📊 연령그룹 통계:")
26 print(age_stats.sort_values('클릭률', ascending=False))
27
28 # 성별 x 연령그룹 교차 분석
29 cross_analysis = df.groupby(['gender', 'age_group']).agg({
30     'clicked': ['count', 'sum', 'mean']
31 }).round(4)
32 cross_analysis.columns = ['총_샘플수', '클릭수', '클릭률']
33
34 print(f"\n📊 성별 x 연령그룹 교차 분석 (클릭률 상위 10개):")
35 print(cross_analysis.sort_values('클릭률', ascending=False).head(10))
36
37 # 사용자 세그먼트 시각화
38 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
39
40 # 1. 성별 분포
41 gender_counts = df['gender'].value_counts()
```

```

42 axes[0,0].pie(gender_counts.values, labels=[f'Gender {i}' for i in gender_counts.index],
43               autopct='%1.1f%%', startangle=90)
44 axes[0,0].set_title('성별 분포', fontsize=12, fontweight='bold')
45
46 # 2. 연령그룹 분포
47 age_counts = df['age_group'].value_counts().sort_index()
48 age_counts.plot(kind='bar', ax=axes[0,1], color='lightcoral', alpha=0.8)
49 axes[0,1].set_title('연령그룹 분포', fontsize=12, fontweight='bold')
50 axes[0,1].set_xlabel('연령그룹')
51 axes[0,1].set_ylabel('샘플 수')
52 axes[0,1].tick_params(axis='x', rotation=45)
53
54 # 3. 성별 클릭률 비교
55 gender_stats['클릭률'].plot(kind='bar', ax=axes[0,2], color='steelblue', alpha=0.8)
56 axes[0,2].set_title('성별 클릭률 비교', fontsize=12, fontweight='bold')
57 axes[0,2].set_xlabel('성별')
58 axes[0,2].set_ylabel('클릭률')
59 axes[0,2].tick_params(axis='x', rotation=0)
60
61 # 4. 연령그룹 클릭률 비교
62 age_stats['클릭률'].plot(kind='bar', ax=axes[1,0], color='green', alpha=0.8)
63 axes[1,0].set_title('연령그룹별 클릭률 비교', fontsize=12, fontweight='bold')
64 axes[1,0].set_xlabel('연령그룹')
65 axes[1,0].set_ylabel('클릭률')
66 axes[1,0].tick_params(axis='x', rotation=45)
67
68 # 5. 성별 x 연령그룹 히트맵 (클릭률)
69 pivot_cross = df.groupby(['gender', 'age_group'])['clicked'].mean().unstack(fill_value=0)
70 in2 = axes[1,1].imshow(pivot_cross.values, cmap='Blues', aspect='auto')
71 axes[1,1].set_title('성별 x 연령그룹 클릭률 히트맵', fontsize=12, fontweight='bold')
72 axes[1,1].set_xlabel('연령그룹')
73 axes[1,1].set_ylabel('성별')
74 axes[1,1].set_yticks(range(len(pivot_cross.index)))
75 axes[1,1].set_yticklabels([f'Gender {i}' for i in pivot_cross.index])
76 axes[1,1].set_xticks(range(len(pivot_cross.columns)))
77 axes[1,1].set_xticklabels(pivot_cross.columns)
78 plt.colorbar(in2, ax=axes[1,1], label='클릭률')
79
80 # 6. 세그먼트별 샘플 수 비교
81 cross_samples = df.groupby(['gender', 'age_group']).size().sort_values(ascending=False).head(10)
82 cross_samples.plot(kind='barh', ax=axes[1,2], color='purple', alpha=0.7)
83 axes[1,2].set_title('세그먼트별 샘플 수 TOP 10', fontsize=12, fontweight='bold')
84 axes[1,2].set_xlabel('샘플 수')
85
86 plt.tight_layout()
87 plt.show()
88
89 # 세그먼트 인사이트 출력
90 print(f"\n🔍 세그먼트 인사이트:")
91 best_gender = gender_stats['클릭률'].idxmax()
92 best_age = age_stats['클릭률'].idxmax()
93 best_segment = cross_analysis['클릭률'].idxmax()
94
95 print(f" 최고 클릭률 성별: Gender {best_gender} ({gender_stats.loc[best_gender, '클릭률']:.4f})")
96 print(f" 최고 클릭률 연령: Age {best_age} ({age_stats.loc[best_age, '클릭률']:.4f})")
97 print(f" 최고 클릭률 세그먼트: Gender {best_segment[0]}, Age {best_segment[1]} ({cross_analysis.loc[best_segment, '클릭률']:.4f})")
98

```

📊 사용자 세그먼트 분석

📊 성별 통계:

	총_샘플수	클릭수	클릭률
gender			
1.0	6713514	138671	0.0195
2.0	3973457	73102	0.0184

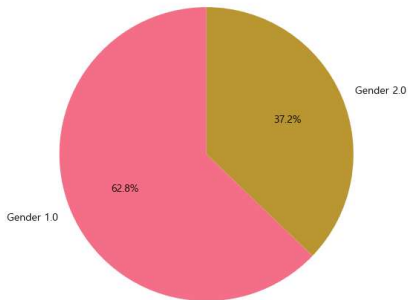
📊 연령그룹 통계:

	총_샘플수	클릭수	클릭률
age_group			
1.0	28963	1947	0.0672
8.0	2646736	60655	0.0229
2.0	358917	8028	0.0224
3.0	479087	9247	0.0193
4.0	554870	10355	0.0187
7.0	3706082	64827	0.0175
5.0	591501	10282	0.0174
6.0	2320895	38432	0.0166

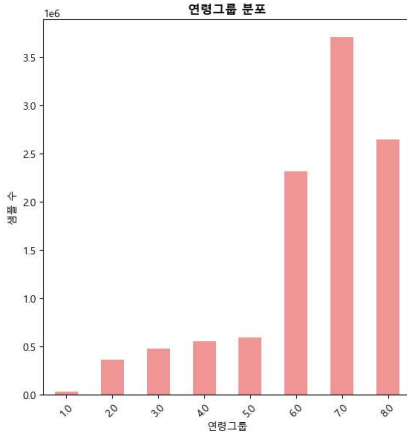
📊 성별 x 연령그룹 교차 분석 (클릭률 상위 10개):

	총_샘플수	클릭수	클릭률
gender age_group			
1.0 1.0	17665	1227	0.0695
2.0 1.0	11290	720	0.0637
2.0 2.0	151170	3774	0.0250
1.0 8.0	1784473	41617	0.0233
2.0 8.0	862263	19838	0.0221
3.0	207356	4326	0.0209
1.0 2.0	207747	4254	0.0205
2.0 4.0	231380	4567	0.0197
1.0 7.0	2398289	43647	0.0182
3.0	271731	4921	0.0181

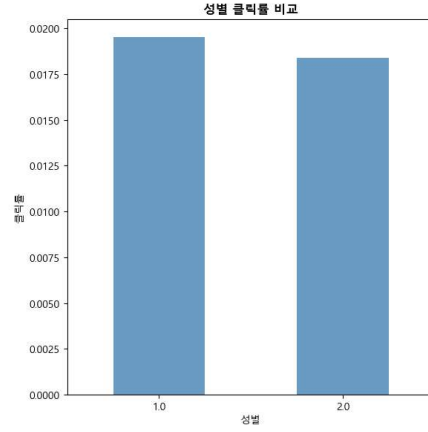
성별 분포



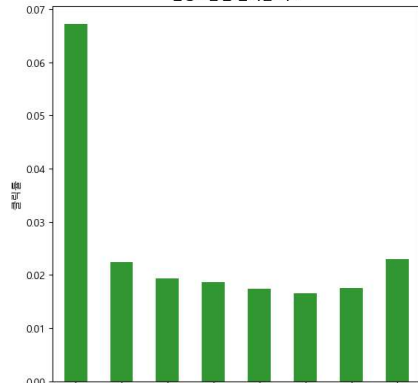
연령그룹 분포



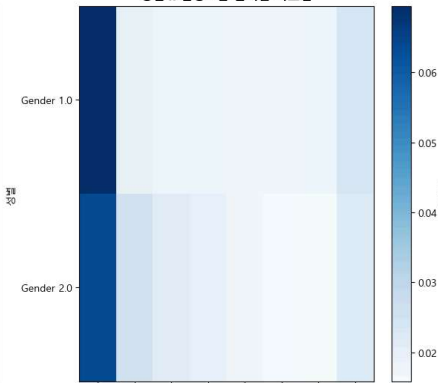
성별 클릭률 비교



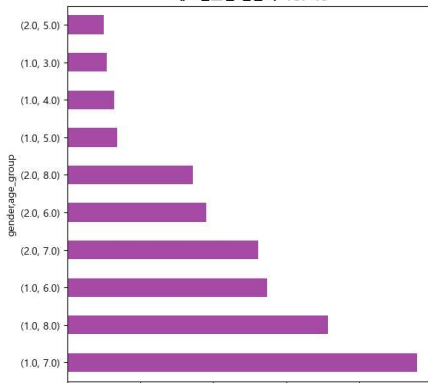
연령그룹별 클릭률 비교



성별 x 연령그룹 클릭률 히트맵



세그먼트별 샘플 수 TOP 10



```
1 # 광고 클릭 예측 데이터 분석 - Part 2
2 # 시퀀스 분석 + 피쳐 상관관계 분석
3
4 import pandas as pd
5 import numpy as np
6 # import matplotlib.pyplot as plt
7 import seaborn as sns
8 from sklearn.preprocessing import LabelEncoder
9 from scipy.stats import chi2_contingency, pearsonr
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # 한글 폰트 설정
14 # plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial Unicode MS', 'Malgun Gothic']
15 # plt.rcParams['axes.unicode_minus'] = False
16 # plt.style.use('default')
17 # sns.set_palette("husl")
18
19 print("🔍 Part 2: 시퀀스 분석 + 피쳐 상관관계 분석")
20 print("=" * 60)
21
22 # 데이터 로드
23 df = pd.read_parquet('./train.parquet')
24 print(f"📄 로드된 데이터: {len(df):,} 샘플")
25
26 # =====
27 # 📁 1. 시퀀스 데이터 (seq) 분석
28 # =====
29
30 print("\n" + "=" * 60)
31 print("📁 시퀀스 데이터 (seq) 분석")
32 print("=" * 60)
33
34 # 시퀀스 길이 분석
35 df['seq_length'] = df['seq'].str.split(',').str.len()
36 df['seq_items'] = df['seq'].str.split(',')
37
38 print("🔍 시퀀스 길이 통계:")
39 seq_stats = df['seq_length'].describe()
40 print(seq_stats)
41
42 # 시퀀스 길이별 클릭률 분석
43 seq_length_analysis = df.groupby('seq_length').agg([
44     'clicked': ['count', 'sum', 'mean']
45 ]).round(4)
46 seq_length_analysis.columns = ['샘플수', '클릭수', '클릭률']
47
48 print(f"📄 시퀀스 길이별 클릭률 (상위 10개):")
```

```

49 print(seq_length_analysis.sort_values('클릭률', ascending=False).head(10))
50
51 # 시퀀스 길이 구간별 분석
52 df['seq_length_bin'] = pd.cut(df['seq_length'],
53                               bins=[0, 100, 300, 600, 1000, float('inf')],
54                               labels=['매우짧음(<100)', '짧음(100-300)', '보통(300-600)', '김(600-1000)', '매우김(1000+)'])
55
56 seq_bin_analysis = df.groupby('seq_length_bin').agg({
57     'clicked': ['count', 'sum', 'mean']
58 }).round(4)
59 seq_bin_analysis.columns = ['생플수', '클릭수', '클릭률']
60
61 print(f"\n📊 시퀀스 길이 구간별 분석:")
62 print(seq_bin_analysis)
63
64 # 시퀀스 내 고유 아이템 수 분석
65 def count_unique_items(seq_str):
66     try:
67         items = seq_str.split(',')
68         return len(set(items))
69     except:
70         return 0
71
72 df['unique_items'] = df['seq'].apply(count_unique_items)
73 df['diversity_ratio'] = df['unique_items'] / df['seq_length']
74
75 print(f"\n🌈 시퀀스 다양성 분석:")
76 print(f"    평균 고유 아이템 수: {df['unique_items'].mean():.2f}")
77 print(f"    평균 다양성 비율: {df['diversity_ratio'].mean():.4f}")
78
79 diversity_analysis = df.groupby(pd.cut(df['diversity_ratio'], bins=5)).agg({
80     'clicked': ['count', 'mean']
81 }).round(4)
82 diversity_analysis.columns = ['생플수', '클릭률']
83 print(f"\n🌈 다양성 비율별 클릭률:")
84 print(diversity_analysis)
85
86 # 시퀀스 분석 시각화
87 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
88
89 # 1. 시퀀스 길이 분포
90 df['seq_length'].hist(bins=50, ax=axes[0,0], color='skyblue', alpha=0.7, edgecolor='black')
91 axes[0,0].set_title('시퀀스 길이 분포', fontsize=12, fontweight='bold')
92 axes[0,0].set_xlabel('시퀀스 길이')
93 axes[0,0].set_ylabel('빈도')
94 axes[0,0].axvline(df['seq_length'].mean(), color='red', linestyle='--', label=f'평균: {df["seq_length"].mean():.0f}')
95 axes[0,0].legend()
96
97 # 2. 시퀀스 길이 vs 클릭률 (구간별)
98 seq_bin_analysis['클릭률'].plot(kind='bar', ax=axes[0,1], color='lightcoral', alpha=0.8)
99 axes[0,1].set_title('시퀀스 길이 구간별 클릭률', fontsize=12, fontweight='bold')
100 axes[0,1].set_xlabel('시퀀스 길이 구간')
101 axes[0,1].set_ylabel('클릭률')
102 axes[0,1].tick_params(axis='x', rotation=45)
103
104 # 3. 다양성 비율 분포
105 df['diversity_ratio'].hist(bins=50, ax=axes[0,2], color='lightgreen', alpha=0.7, edgecolor='black')
106 axes[0,2].set_title('시퀀스 다양성 비율 분포', fontsize=12, fontweight='bold')
107 axes[0,2].set_xlabel('다양성 비율')
108 axes[0,2].set_ylabel('빈도')
109 axes[0,2].axvline(df['diversity_ratio'].mean(), color='red', linestyle='--', label=f'평균: {df["diversity_ratio"].mean():.3f}')
110 axes[0,2].legend()
111
112 # 4. 시퀀스 길이 vs 클릭률 산점도
113 sample_df = df.sample(n=min(10000, len(df))) # 샘플링으로 시각화 최적화
114 clicked_color = sample_df['clicked'].map({0: 'lightblue', 1: 'red'})
115 axes[1,0].scatter(sample_df['seq_length'], sample_df['diversity_ratio'],
116                  c=clicked_color, alpha=0.6, s=10)
117 axes[1,0].set_title('시퀀스 길이 vs 다양성 (클릭률 색상)', fontsize=12, fontweight='bold')
118 axes[1,0].set_xlabel('시퀀스 길이')
119 axes[1,0].set_ylabel('다양성 비율')
120
121 # 5. 고유 아이템 수 vs 클릭률
122 unique_bins = pd.cut(df['unique_items'], bins=10)
123 unique_analysis = df.groupby(unique_bins)['clicked'].mean()
124 unique_analysis.plot(kind='line', ax=axes[1,1], marker='o', color='purple', linewidth=2)
125 axes[1,1].set_title('고유 아이템 수별 클릭률', fontsize=12, fontweight='bold')
126 axes[1,1].set_xlabel('고유 아이템 수 구간')
127 axes[1,1].set_ylabel('클릭률')
128 axes[1,1].tick_params(axis='x', rotation=45)
129 axes[1,1].grid(True, alpha=0.3)
130
131 # 6. 시퀀스 길이와 클릭률의 관계 (박스플롯)
132 df['seq_length_category'] = pd.cut(df['seq_length'], bins=5, labels=['매우짧음', '짧음', '보통', '김', '매우김'])
133 df.boxplot(column='clicked', by='seq_length_category', ax=axes[1,2])
134 axes[1,2].set_title('시퀀스 길이별 클릭률 분포', fontsize=12, fontweight='bold')
135 axes[1,2].set_xlabel('시퀀스 길이 카테고리')
136 axes[1,2].set_ylabel('클릭률')
137
138 plt.suptitle('', fontsize=16) # 기본 제목 제거
139 plt.tight_layout()
140 plt.show()
141

```

🔍 Part 2: 시퀀스 분석 + 피처 상관관계 분석

📊 로드된 데이터: 10,784,179 샘플

🔗 시퀀스 데이터 (seq) 분석

```

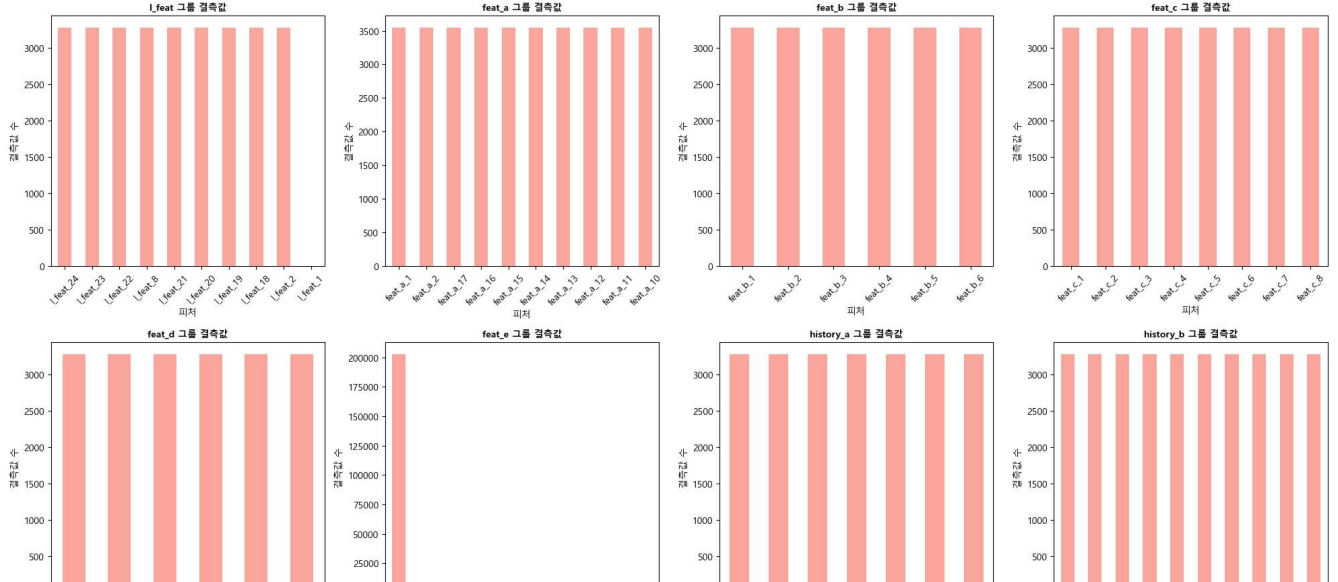
1
2 # =====
3 # 📊 2. 피쳐 그룹별 분석
4 # =====
5
6 print("\n" + "="*60)
7 print("📊 피쳐 그룹별 분석")
8 print("="*60)
9
10 # 피쳐 그룹 정의
11 feature_groups = {
12     'l_feat': [col for col in df.columns if col.startswith('l_feat_')],
13     'feat_a': [col for col in df.columns if col.startswith('feat_a_')],
14     'feat_b': [col for col in df.columns if col.startswith('feat_b_')],
15     'feat_c': [col for col in df.columns if col.startswith('feat_c_')],
16     'feat_d': [col for col in df.columns if col.startswith('feat_d_')],
17     'feat_e': [col for col in df.columns if col.startswith('feat_e_')],
18     'history_a': [col for col in df.columns if col.startswith('history_a_')],
19     'history_b': [col for col in df.columns if col.startswith('history_b_')]
20 }
21
22 print("🔍 피쳐 그룹별 구성:")
23 for group, features in feature_groups.items():
24     missing_count = df[features].isnull().sum().sum()
25     print(f"    {group}: {len(features)}개 피쳐, 결측값: {missing_count:,}개")
26
27 # 피쳐 그룹별 결측값 패턴 분석
28 fig, axes = plt.subplots(2, 4, figsize=(20, 10))
29 axes = axes.flatten()
30
31 for i, (group, features) in enumerate(feature_groups.items()):
32     if i < len(axes):
33         missing_pattern = df[features].isnull().sum().sort_values(ascending=False)
34         if len(missing_pattern) > 0:
35             missing_pattern.head(10).plot(kind='bar', ax=axes[i], color='salmon', alpha=0.7)

```

```
36 axes[1].set_title(f'{group} 그룹 결속값', fontsize=10, fontweight='bold')
37 axes[1].set_xlabel('피처')
38 axes[1].set_ylabel('결속값 수')
39 axes[1].tick_params(axis='x', rotation=45)
40
41 plt.tight_layout()
42 plt.show()
43
```

피처 그룹별 분석

피처 그룹별 구성:
l_feat: 27개 피처, 결속값: 29,502개
feat_a: 18개 피처, 결속값: 63,900개
feat_b: 6개 피처, 결속값: 19,668개
feat_c: 8개 피처, 결속값: 26,224개
feat_d: 6개 피처, 결속값: 19,668개
feat_e: 10개 피처, 결속값: 232,190개
history_a: 7개 피처, 결속값: 22,946개
history_b: 30개 피처, 결속값: 98,340개



```
1
2 #
3 # 3. 타겟과 피치간 상관관계 분석
4 #
5
6 print("\n" + "="*60)
7 print(" 3. 타겟과 피치간 상관관계 분석")
8 print("="*60)
9
10 # 수치형 피쳐들과 타겟의 상관관계
11 numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
12 numeric_features = [col for col in numeric_features if col != 'clicked']
13
14 # 상관관계 계산 (결속값 처리)
15 correlations = []
16 for feature in numeric_features:
17     if df[feature].notna().sum() > 100: # 충분한 데이터가 있는 경우만
18         corr = df[['clicked', feature]].corr().iloc[0, 1]
19         if not np.isnan(corr):
20             correlations.append((feature, abs(corr), corr))
21
22 correlations.sort(key=lambda x: x[1], reverse=True)
23
24 print("\n 3. 타겟과 상관관계 TOP 20:")
25 for i, (feature, abs_corr, corr) in enumerate(correlations[:20]):
26     print(f" {i+1:2d}. {feature:20s}: {corr:8.4f} (절댓값: {abs_corr:.4f})")
27
28 # 상관관계 시각화
29 top_features = [item[0] for item in correlations[:20]]
30 corr_values = [item[2] for item in correlations[:20]]
31
32 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
33
34 # 1. 상위 20개 피쳐의 상관관계
35 axes[0,0].barh(range(len(top_features)), corr_values, color='red' if x < 0 else 'blue' for x in corr_values, alpha=0.7)
36 axes[0,0].set_yticks(range(len(top_features)))
37 axes[0,0].set_yticklabels(top_features)
38 axes[0,0].set_title('타겟과 상관관계 TOP 20', fontsize=12, fontweight='bold')
39 axes[0,0].set_xlabel('상관계수')
40 axes[0,0].axvline(0, color='black', linestyle='--', alpha=0.3)
41
42 # 2. 피쳐 그룹별 평균 상관관계
43 group_correlations = {}
44 for group, features in feature_groups.items():
45     group_corrs = []
46     for feature in features:
47         if feature in [item[0] for item in correlations]:
48             corr_val = next(item[2] for item in correlations if item[0] == feature)
49             group_corrs.append(abs(corr_val))
50     if group_corrs:
51         group_correlations[group] = np.mean(group_corrs)
52
53 if group_correlations:
54     groups = list(group_correlations.keys())
55     values = list(group_correlations.values())
56     axes[0,1].bar(groups, values, color='green', alpha=0.7)
57     axes[0,1].set_title('피쳐 그룹별 평균 상관관계', fontsize=12, fontweight='bold')
58     axes[0,1].set_xlabel('피쳐 그룹')
59     axes[0,1].set_ylabel('평균 절댓값 상관계수')
60     axes[0,1].tick_params(axis='x', rotation=45)
61
62 # 3. 상관관계 분포
63 all_corr_values = [item[2] for item in correlations]
64 axes[1,0].hist(all_corr_values, bins=30, color='purple', alpha=0.7, edgecolor='black')
65 axes[1,0].set_title('상관계수 분포', fontsize=12, fontweight='bold')
66 axes[1,0].set_xlabel('상관계수')
67 axes[1,0].set_ylabel('빈도')
68 axes[1,0].axvline(0, color='red', linestyle='--', label='0')
69 axes[1,0].legend()
70
71 # 4. 시간/사용자 피치와의 상관관계
72 time_user_features = ['hour', 'day_of_week', 'gender', 'age_group', 'inventory_id', 'seq_length', 'unique_items', 'diversity_ratio']
73 time_user_corrs = []
74 time_user_names = []
75
76 for feature in time_user_features:
77     if feature in df.columns:
```

```

78     corr = df[['clicked', feature]].corr().iloc[0, 1]
79     if not np.isnan(corr):
80         time_user_corrs.append(corr)
81         time_user_names.append(feature)
82
83 if time_user_corrs:
84     axes[1,1].barh(range(len(time_user_names)), time_user_corrs,
85                    color=['red' if x < 0 else 'blue' for x in time_user_corrs], alpha=0.7)
86     axes[1,1].set_yticks(range(len(time_user_names)))
87     axes[1,1].set_yticklabels(time_user_names)
88     axes[1,1].set_title('시간/사용자/시퀀스 피쳐 상관관계', fontsize=12, fontweight='bold')
89     axes[1,1].set_xlabel('상관계수')
90     axes[1,1].axvline(0, color='black', linestyle='-', alpha=0.3)
91
92 plt.tight_layout()
93 plt.show()
94

```

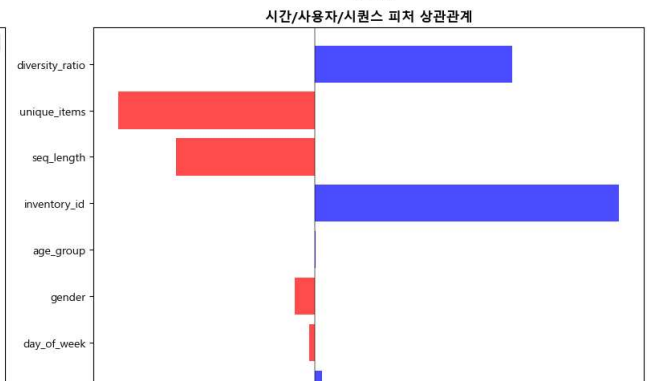
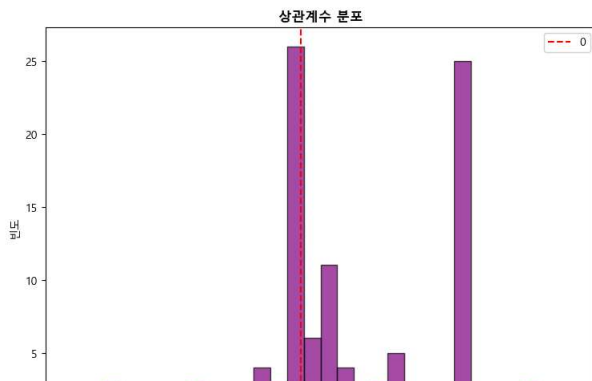
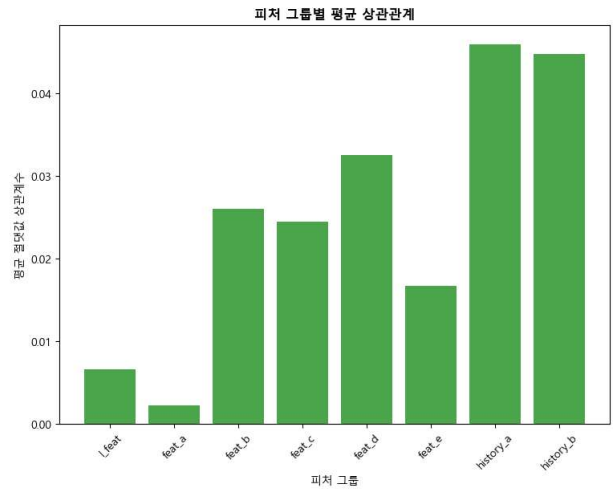
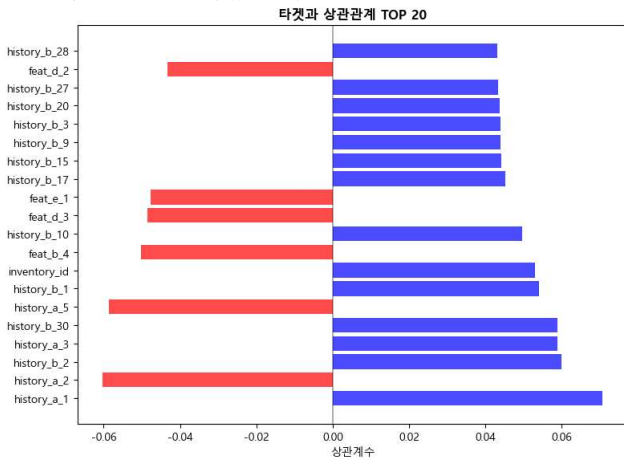
타겟과 피쳐간 상관관계 분석

타겟과 상관관계 TOP 20:

```

1. history_a_1      :  0.0707 (절댓값: 0.0707)
2. history_a_2      : -0.0603 (절댓값: 0.0603)
3. history_b_2      :  0.0600 (절댓값: 0.0600)
4. history_a_3      :  0.0589 (절댓값: 0.0589)
5. history_b_30     :  0.0588 (절댓값: 0.0588)
6. history_a_5      : -0.0586 (절댓값: 0.0586)
7. history_b_1      :  0.0541 (절댓값: 0.0541)
8. inventory_id     :  0.0529 (절댓값: 0.0529)
9. feat_b_4         : -0.0501 (절댓값: 0.0501)
10. history_b_10    :  0.0496 (절댓값: 0.0496)
11. feat_d_3        : -0.0484 (절댓값: 0.0484)
12. feat_e_1        : -0.0476 (절댓값: 0.0476)
13. history_b_17    :  0.0452 (절댓값: 0.0452)
14. history_b_15    :  0.0441 (절댓값: 0.0441)
15. history_b_9     :  0.0441 (절댓값: 0.0441)
16. history_b_3     :  0.0439 (절댓값: 0.0439)
17. history_b_20    :  0.0438 (절댓값: 0.0438)
18. history_b_27    :  0.0434 (절댓값: 0.0434)
19. feat_d_2        : -0.0433 (절댓값: 0.0433)
20. history_b_28    :  0.0432 (절댓값: 0.0432)

```



```

1
2 # =====
3 # 🍌 4. 피쳐 중요도 분석 (간단한 통계적 방법)
4 # =====
5
6 print("\n" + "="*60)
7 print("🍌 피쳐 중요도 분석")
8 print("="*60)
9
10 # 범주형 피쳐의 카이제곱 검정
11 categorical_features = ['gender', 'age_group', 'day_of_week', 'hour', 'inventory_id']
12 chi2_results = []
13
14 for feature in categorical_features:
15     if feature in df.columns:
16         # 결측값 제거
17         temp_df = df[['feature', 'clicked']].dropna()
18         if len(temp_df) > 0:
19             contingency = pd.crosstab(temp_df[feature], temp_df['clicked'])
20             if contingency.shape[0] > 1 and contingency.shape[1] > 1:
21                 chi2, p_value, dof, expected = chi2_contingency(contingency)
22                 chi2_results.append((feature, chi2, p_value))
23
24 chi2_results.sort(key=lambda x: x[1], reverse=True)
25
26 print("\n 범주형 피쳐 카이제곱 검정 결과:")
27 for feature, chi2_stat, p_val in chi2_results:
28     significance = "" if p_val < 0.001 else "*" if p_val < 0.01 else "**" if p_val < 0.05 else ""
29     print(f" {feature:15s}: χ² = {chi2_stat:10.2f}, p-value = {p_val:.6f} {significance}")
30
31 # 수치형 피쳐의 평균 차이 분석
32 print(f"\n 수치형 피쳐 클릭/비클릭 평균 차이 (TOP 15):")
33 feature_diffs = []
34
35 for feature in top_features[:15]: # 상관관계 높은 피쳐들만
36     clicked_mean = df[df['clicked'] == 1][feature].mean()
37     not_clicked_mean = df[df['clicked'] == 0][feature].mean()
38     diff = clicked_mean - not_clicked_mean

```

```
39 feature_diffs.append((feature, clicked_mean, not_clicked_mean, diff))
40
41 feature_diffs.sort(key=lambda x: abs(x[3]), reverse=True)
42
43 for feature, click_mean, no_click_mean, diff in feature_diffs:
44     print(f"    {feature:20s}: 클릭 {click_mean:8.4f} vs 비클릭 {no_click_mean:8.4f} (차이: {diff:8.4f})")
45
```

🔥 피처 중요도 분석

```
📊 범주형 피처 카이제곱 검정 결과:
inventory_id    :  $\chi^2$  = 8744.28, p-value = 0.000000 ***
age_group       :  $\chi^2$  = 1389.78, p-value = 0.000000 ***
day_of_week     :  $\chi^2$  = 176.78, p-value = 0.000000 ***
hour            :  $\chi^2$  = 143.95, p-value = 0.000000 ***
gender          :  $\chi^2$  = 25.02, p-value = 0.000001 ***

📊 수치형 피처 클릭/비클릭 평균 차이 (TOP 15):
inventory_id    : 클릭 40.1495 vs 비클릭 31.4623 (차이: 0.6872)
feat_a_1        : 클릭 57.7288 vs 비클릭 61.1789 (차이: -3.4581)
history_b_2     : 클릭 1.4913 vs 비클릭 0.5867 (차이: 0.9046)
history_b_1     : 클릭 0.9465 vs 비클릭 0.4187 (차이: 0.5278)
history_b_30    : 클릭 0.5383 vs 비클릭 0.2112 (차이: 0.3270)
history_b_10    : 클릭 0.3683 vs 비클릭 0.1490 (차이: 0.2193)
feat_d_3        : 클릭 1.2786 vs 비클릭 1.4781 (차이: -0.1995)
history_a_1     : 클릭 0.3069 vs 비클릭 0.1100 (차이: 0.1969)
history_b_17    : 클릭 0.2566 vs 비클릭 0.1036 (차이: 0.1530)
history_a_2     : 클릭 -0.2286 vs 비클릭 -0.1082 (차이: -0.1204)
feat_b_4        : 클릭 0.6640 vs 비클릭 0.7719 (차이: -0.1078)
history_a_3     : 클릭 0.0969 vs 비클릭 0.0487 (차이: 0.0483)
history_b_9     : 클릭 0.0718 vs 비클릭 0.0293 (차이: 0.0425)
history_a_5     : 클릭 -0.0313 vs 비클릭 -0.0147 (차이: -0.0166)
history_b_15    : 클릭 0.0120 vs 비클릭 0.0049 (차이: 0.0071)
```

```
1 # 광고 클릭 예측 데이터 분석 - Part 3
2 # 고급 분석: PCA/클러스터링 + 심화 인사이트 + 인터랙티브 분석
3
4 import pandas as pd
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from sklearn.decomposition import PCA
9 from sklearn.cluster import KMeans
10 from sklearn.preprocessing import StandardScaler, LabelEncoder
11 from sklearn.manifold import TSNE
12 from scipy.stats import chi2_contingency
13 import warnings
14 warnings.filterwarnings('ignore')
15
16 # # 한글 폰트 설정
17 plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial Unicode MS', 'Malgun Gothic']
18 plt.rcParams['axes.unicode_minus'] = False
19 plt.style.use('default')
20 sns.set_palette("husl")
21
22 print(f"🔥 Part 3: 고급 분석 - PCA/클러스터링 + 심화 인사이트")
23 print(f"=" * 70)
24
25
26 # 시퀀스 분석 결과 재구성 (Part 2에서 생성한 피치들)
27 df['seq_length'] = df['seq'].str.split(',').str.len()
28 def count_unique_items(seq_str):
29     try:
30         items = seq_str.split(',')
31         return len(set(items))
32     except:
33         return 0
34
35 df['unique_items'] = df['seq'].apply(count_unique_items)
36 df['diversity_ratio'] = df['unique_items'] / df['seq_length']
37
38 print(f"📊 로드된 데이터: {len(df):,} 샘플")
39
40 # =====
41 # 🐡 1. 피처 상호작용 심화 분석
42 # =====
43
44 print(f"\n" + "="*70)
45 print(f"🐡 피처 상호작용 심화 분석")
46 print(f"="*70)
47
48 # 핵심 피치들 정의 (Part 2 결과 기반)
49 top_features = [
50     'history_a_1', 'history_a_2', 'history_b_2', 'history_a_3', 'history_b_30',
51     'history_a_5', 'history_b_1', 'inventory_id', 'feat_b_4', 'history_b_10'
52 ]
53
54 # 시간 + 사용자 + 시퀀스 피치
55 context_features = ['hour', 'day_of_week', 'gender', 'age_group', 'seq_length', 'diversity_ratio']
56
57 # 상호작용 분석: inventory_id x 시간대
58 print(f"📊 inventory_id x 시간대 상호작용:")
59 inv_hour_analysis = df.groupby(['inventory_id', 'hour']).agg({
60     'clicked': ['count', 'sum', 'mean']
61 }).round(4)
62 inv_hour_analysis.columns = ['샘플수', '클릭수', '클릭률']
63
64 # 상위 inventory_id들만 분석
65 top_inventories = df['inventory_id'].value_counts().head(10).index
66 inv_hour_filtered = inv_hour_analysis.loc[top_inventories]
67 best_combinations = inv_hour_filtered.sort_values('클릭률', ascending=False).head(10)
68 print(f"최고 성과 조합 (inventory_id x hour):")
69 print(best_combinations)
70
71 # 연평균 x 다양성 비율 상호작용
72 print(f"📊 연평균 x 다양성 비율 상호작용:")
73 df['diversity_bin'] = pd.cut(df['diversity_ratio'], bins=4, labels=['낮음', '보통', '높음', '매우높음'])
74 age_diversity_analysis = df.groupby(['age_group', 'diversity_bin']).agg({
75     'clicked': ['count', 'mean']
76 }).round(4)
77 age_diversity_analysis.columns = ['샘플수', '클릭률']
78 print(age_diversity_analysis.sort_values('클릭률', ascending=False).head(10))
79
80 # 상호작용 시각화
81 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
82
83 # 1. inventory_id TOP 10의 시간별 클릭률 히트맵
84 top_inv_hour = df[df['inventory_id'].isin(top_inventories)]
85 pivot_inv_hour = top_inv_hour.groupby(['inventory_id', 'hour'])[ 'clicked'].mean().unstack(fill_value=0)
86 im1 = axes[0,0].imshow(pivot_inv_hour.values, cmap='YlOrRd', aspect='auto')
87 axes[0,0].set_title('상위 지면 x 시간 클릭률 히트맵', fontsize=12, fontweight='bold')
88 axes[0,0].set_xlabel('시간')
89 axes[0,0].set_ylabel('지면 ID')
90 axes[0,0].set_yticks(range(len(pivot_inv_hour.index)))
91 axes[0,0].set_yticklabels(pivot_inv_hour.index)
92 plt.colorbar(im1, ax=axes[0,0], label='클릭률')
93
94 # 2. 연평균 x 다양성 히트맵
95 pivot_age_div = df.groupby(['age_group', 'diversity_bin'])[ 'clicked'].mean().unstack(fill_value=0)
96 im2 = axes[0,1].imshow(pivot_age_div.values, cmap='Blues', aspect='auto')
97 axes[0,1].set_title('연평균 x 다양성 클릭률 히트맵', fontsize=12, fontweight='bold')
98 axes[0,1].set_xlabel('다양성 수준')
99 axes[0,1].set_ylabel('연평균')
100 axes[0,1].set_yticks(range(len(pivot_age_div.index)))
101 axes[0,1].set_yticklabels(pivot_age_div.index)
102 axes[0,1].set_xticks(range(len(pivot_age_div.columns)))
103 axes[0,1].set_xticklabels(pivot_age_div.columns)
```

```

104 plt.colorbar(im2, ax=axes[0,1], label='클릭률')
105
106 # 3. 시퀀스 길이 vs 다양성 vs 클릭률 (3D 스타일)
107 sample_df = df.sample(n=10000) # 샘플링
108 sample_df = sample_df[sample_df['clicked'] == 1]
109 scatter = axes[0,2].scatter(sample_df['seq_length'], sample_df['diversity_ratio'],
110                             c=sample_df['clicked'], cmap='Greens', alpha=0.6, s=10)
111 axes[0,2].set_title('시퀀스 길이 vs 다양성 vs 클릭', fontsize=12, fontweight='bold')
112 axes[0,2].set_xlabel('시퀀스 길이')
113 axes[0,2].set_ylabel('다양성 비율')
114 plt.colorbar(scatter, ax=axes[0,2], label='클릭 여부')
115
116 # 4. 성별 x 시간 x 클릭률
117 pivot_gender_hour = df.groupby(['gender', 'hour'])['clicked'].mean().unstack(fill_value=0)
118 im3 = axes[1,0].imshow(pivot_gender_hour.values, cmap='Greens', aspect='auto')
119 axes[1,0].set_title('성별 x 시간 클릭률 히트맵', fontsize=12, fontweight='bold')
120 axes[1,0].set_xlabel('시간')
121 axes[1,0].set_ylabel('성별')
122 axes[1,0].set_yticks(range(len(pivot_gender_hour.index)))
123 axes[1,0].set_yticklabels(['Gender {int(x)}' for x in pivot_gender_hour.index])
124 plt.colorbar(im3, ax=axes[1,0], label='클릭률')
125
126 # 5. 요일 x 연령그룹 패턴
127 pivot_day_age = df.groupby(['day_of_week', 'age_group'])['clicked'].mean().unstack(fill_value=0)
128 im4 = axes[1,1].imshow(pivot_day_age.values, cmap='Purples', aspect='auto')
129 axes[1,1].set_title('요일 x 연령그룹 클릭률 히트맵', fontsize=12, fontweight='bold')
130 axes[1,1].set_ylabel('연령그룹')
131 axes[1,1].set_xlabel('요일')
132 day_names = ['월', '화', '수', '목', '금', '토', '일']
133 axes[1,1].set_yticks(range(len(pivot_day_age.index)))
134 axes[1,1].set_yticklabels([day_names[i] if i < len(day_names) else f'Day_{i}' for i in pivot_day_age.index])
135 plt.colorbar(im4, ax=axes[1,1], label='클릭률')
136
137 # 6. 종합 상호작용 강도 분석
138 interaction_strength = []
139 feature_pairs = [
140     ('inventory_id', 'hour'), ('age_group', 'diversity_ratio'),
141     ('gender', 'hour'), ('day_of_week', 'age_group'),
142     ('inventory_id', 'age_group'), ('seq_length', 'diversity_ratio')
143 ]
144
145 for feat1, feat2 in feature_pairs:
146     if feat1 in df.columns and feat2 in df.columns:
147         cross_tab = pd.crosstab(df[feat1], df[feat2], df['clicked'], aggfunc='mean')
148         variance = cross_tab.var().var() if not cross_tab.empty else 0
149         interaction_strength.append((f'{feat1} x {feat2}', variance))
150
151 interaction_strength.sort(key=lambda x: x[1], reverse=True)
152 features_names = [item[0] for item in interaction_strength]
153 strengths = [item[1] for item in interaction_strength]
154
155 axes[1,2].barh(range(len(features_names)), strengths, color='coral', alpha=0.8)
156 axes[1,2].set_yticks(range(len(features_names)))
157 axes[1,2].set_yticklabels(features_names)
158 axes[1,2].set_title('피쳐 상호작용 강도', fontsize=12, fontweight='bold')
159 axes[1,2].set_xlabel('상호작용 강도 (분산)')
160
161 plt.tight_layout()
162 plt.show()
163

```

로드된 데이터: 2,000,000 샘플

피쳐 상호작용 심화 분석

inventory_id x 시간대 상호작용:

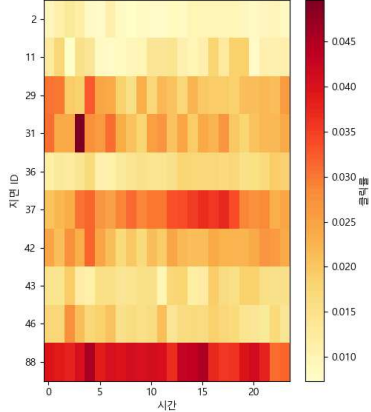
최고 성과 조합 (inventory_id x hour):

inventory_id	hour	샘플수	클릭수	클릭률
31	645	32	0.0496	
88	4	1399	64	0.0457
	15	5913	268	0.0453
	14	6322	274	0.0433
	13	6659	285	0.0428
	20	6816	281	0.0412
	10	6690	274	0.0410
	8	6052	247	0.0408
	11	6498	264	0.0407
	3	1482	57	0.0407

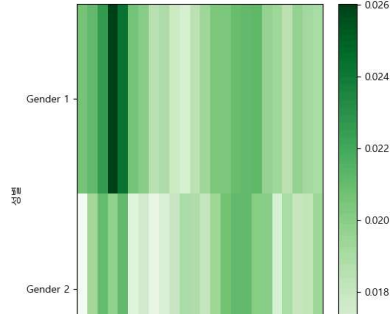
연령그룹 x 다양성 비율 상호작용:

age_group	diversity_bin	샘플수	클릭률
1.0	높음	638	0.1082
	매우높음	545	0.0862
	보통	1889	0.0768
8.0	매우높음	4141	0.0553
	높음	4278	0.0535
1.0	낮음	2304	0.0534
7.0	매우높음	8849	0.0445
6.0	매우높음	8862	0.0367
7.0	높음	10281	0.0357
8.0	보통	35548	0.0334

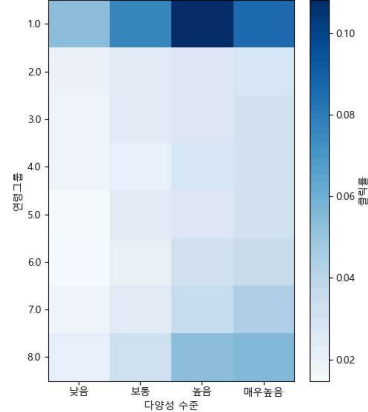
상위 자연 x 시간 클릭률 히트맵



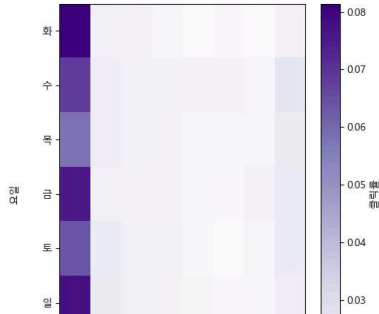
성별 x 시간 클릭률 히트맵



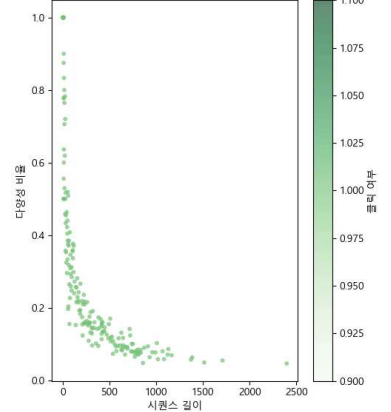
연령그룹 x 다양성 클릭률 히트맵



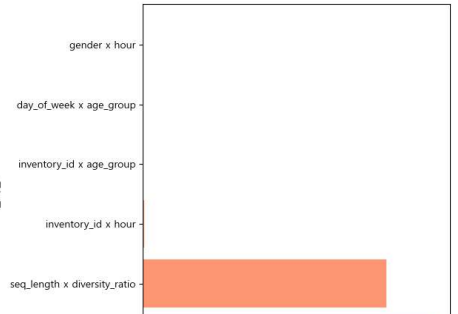
요일 x 연령그룹 클릭률 히트맵



시퀀스 길이 vs 다양성 vs 클릭



피쳐 상호작용 강도



```

1
2 # =====
3 # 📊 2. PCA 차원 축소 및 패턴 발견
4 # =====
5
6 print("\n" + "-"*70)
7 print("📊 PCA 차원 축소 및 패턴 발견")
8 print("-"*70)
9
10 # 수치형 피쳐만 선택하고 결측값 처리
11 numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
12 numeric_cols = [col for col in numeric_cols if col not in ['clicked']]
13
14 # PCA를 위한 데이터 준비
15 pca_data = df[numeric_cols].fillna(df[numeric_cols].median())
16 scaler = StandardScaler()
17 pca_data_scaled = scaler.fit_transform(pca_data)
18
19 # PCA 수행
20 pca = PCA()
21 pca_result = pca.fit_transform(pca_data_scaled)
22
23 # 누적 분산 비율 계산
24 cumsum_var_ratio = np.cumsum(pca.explained_variance_ratio_)
25 n_components_95 = np.argmax(cumsum_var_ratio >= 0.95) + 1
26 n_components_90 = np.argmax(cumsum_var_ratio >= 0.90) + 1
27
28 print(f"📊 PCA 분석 결과:")
29 print(f"   95% 분산 설명하는 성분 수: {n_components_95}")
30 print(f"   90% 분산 설명하는 성분 수: {n_components_90}")
31 print(f"   처음 10개 성분의 분산 설명율: {cumsum_var_ratio[9]:.4f}")
32
33 # 주성분별 피쳐 기여도 분석
34 feature_importance_pc1 = pd.DataFrame({
35     'feature': numeric_cols,
36     'importance': np.abs(pca.components_[0])
37 }).sort_values('importance', ascending=False)
38
39 feature_importance_pc2 = pd.DataFrame({
40     'feature': numeric_cols,
41     'importance': np.abs(pca.components_[1])
42 }).sort_values('importance', ascending=False)
43
44 print(f"\n🔍 PC1 (제1주성분) 주요 기여 피쳐 TOP 10:")
45 print(feature_importance_pc1.head(10))
46
47 print(f"\n🔍 PC2 (제2주성분) 주요 기여 피쳐 TOP 10:")
48 print(feature_importance_pc2.head(10))
49
50 # PCA 시각화
51 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
52
53 # 1. 분산 설명을 스크리 플롯

```

```
54 axes[0,0].plot(range(1, 21), pca.explained_variance_ratio_[0:20], 'bo-', linewidth=2)
55 axes[0,0].set_title('PCA 스크리 플롯 (상위 20개 성분)', fontsize=12, fontweight='bold')
56 axes[0,0].set_xlabel('주성분')
57 axes[0,0].set_ylabel('분산 설명율')
58 axes[0,0].grid(True, alpha=0.3)
59
60 # 2. 누적 분산 설명율
61 axes[0,1].plot(range(1, 51), cumsum_var_ratio[0:50], 'ro-', linewidth=2)
62 axes[0,1].axhline(y=0.95, color='green', linestyle='--', label='95%')
63 axes[0,1].axhline(y=0.90, color='blue', linestyle='--', label='90%')
64 axes[0,1].set_title('누적 분산 설명율', fontsize=12, fontweight='bold')
65 axes[0,1].set_xlabel('주성분 수')
66 axes[0,1].set_ylabel('누적 분산 설명율')
67 axes[0,1].legend()
68 axes[0,1].grid(True, alpha=0.3)
69
70 # 3. PC1 vs PC2 산점도 (클릭별 색상)
71 sample_indices = np.random.choice(len(pca_result), size=5000, replace=False)
72 clicked_colors = df.iloc[sample_indices]['clicked'].map({0: 'lightblue', 1: 'red'})
73 axes[0,2].scatter(pca_result[sample_indices, 0], pca_result[sample_indices, 1],
74                  c=clicked_colors, alpha=0.6, s=10)
75 axes[0,2].set_title('PCA 공간에서 클릭 패턴', fontsize=12, fontweight='bold')
76 axes[0,2].set_xlabel('PC1 (pca.explained_variance_ratio_[0]:.3f)')
77 axes[0,2].set_ylabel('PC2 (pca.explained_variance_ratio_[1]:.3f)')
78
79 # 4. PC1 기여도 상위 피쳐들
80 top_pc1_features = feature_importance_pc1.head(15)
81 axes[1,0].barh(range(len(top_pc1_features)), top_pc1_features['importance'], color='skyblue', alpha=0.8)
82 axes[1,0].set_yticks(range(len(top_pc1_features)))
83 axes[1,0].set_yticklabels(top_pc1_features['feature'])
84 axes[1,0].set_title('PC1 주요 기여 피쳐', fontsize=12, fontweight='bold')
85 axes[1,0].set_xlabel('기여도 (절댓값)')
86
87 # 5. PC2 기여도 상위 피쳐들
88 top_pc2_features = feature_importance_pc2.head(15)
89 axes[1,1].barh(range(len(top_pc2_features)), top_pc2_features['importance'], color='lightcoral', alpha=0.8)
90 axes[1,1].set_yticks(range(len(top_pc2_features)))
91 axes[1,1].set_yticklabels(top_pc2_features['feature'])
92 axes[1,1].set_title('PC2 주요 기여 피쳐', fontsize=12, fontweight='bold')
93 axes[1,1].set_xlabel('기여도 (절댓값)')
94
95 # 6. 주성분별 클릭률 분석
96 pc1_bins = pd.cut(pca_result[:, 0], bins=10)
97 pc1_click_rate = df.groupby(pc1_bins)['clicked'].mean()
98 pc1_click_rate.plot(kind='line', ax=axes[1,2], marker='o', color='purple', linewidth=2)
99 axes[1,2].set_title('PC1 구간별 클릭률', fontsize=12, fontweight='bold')
100 axes[1,2].set_xlabel('PC1 구간')
101 axes[1,2].set_ylabel('클릭률')
102 axes[1,2].tick_params(axis='x', rotation=45)
103 axes[1,2].grid(True, alpha=0.3)
104
105 plt.tight_layout()
106 plt.show()
107
```

PCA 차원축소 및 패턴 발견

PCA 분석 결과:
95% 분산 설명하는 성분 수: 60
90% 분산 설명하는 성분 수: 48
저음 10개 성분의 분산 설명율: 0.6023

PC1 (제1주성분) 주요 기여 피쳐 TOP 10:

	feature	importance
107	history_b_21	0.165217
104	history_b_18	0.165216
102	history_b_16	0.165216
109	history_b_23	0.165215
111	history_b_25	0.165215
115	history_b_29	0.165214
110	history_b_24	0.165213
92	history_b_6	0.165213
97	history_b_11	0.165212
108	history_b_22	0.165210

PC2 (제2주성분) 주요 기여 피쳐 TOP 10:

	feature	importance
68	feat_b_5	0.217926
118	unique_items	0.209113
117	seq_length	0.207913
83	history_a_4	0.206077
85	history_a_6	0.203687
39	feat_e_8	0.201816
86	history_a_7	0.201534
33	feat_e_2	0.198985
58	feat_b_3	0.186632
46	feat_d_5	0.180041

